
SESIÓN 3 DE 6 · 5 HORAS · BLOQUE 5: 2H15 · DESCANSO 30 MIN · BLOQUE 6: 2H15

RESÚMENES, AUTENTICACIÓN Y CONTRASEÑAS

HMAC-SHA256 NO ESTÁ ROTO. ENTONCES, ¿POR QUÉ NO LE SIRVIÓ A COOPABURRÁ?

SIO0010 · Técnicas Criptográficas · Especialización en Seguridad de la Información de las Organizaciones · Facultad de Ingeniería · Institución Universitaria de Envigado

SECCIÓN

01

DONDE QUEDAMOS

Realimentación de la entrega 2 y lo que hoy resolvemos del caso

AGENDA DE LA SESIÓN

Hoy resolvemos los dos hallazgos que hicieron posible el fraude: H1, las contraseñas, y H2, la firma de operaciones. Son los dos que explican por qué alguien pudo transferir dinero ajeno.

BLOQUE	MINUTOS	CONTENIDO	MODALIDAD
Bloque 5	0 – 20	Realimentación de la entrega 2 y hallazgos comunes	Taller
	20 – 80	Funciones resumen: qué prometen, cuáles ya no sirven y para qué sirven	Magistral
	80 – 135	Integridad no es autenticidad: HMAC, HKDF y la pregunta del abogado	Magistral
Descanso	30		
Bloque 6	0 – 60	Contraseñas: derivación, Argon2id, pimienta, llaves de acceso y recuperación	Magistral
	60 – 110	Laboratorio 3: atacar la evidencia B, con el ejercicio en Python	Laboratorio
	110 – 130	Primera evaluación de conocimiento	Evaluación
	130 – 135	Cierre y trabajo independiente	Magistral

LO QUE QUEDÓ DEL JUEVES

01

El modo de operación importa más que el algoritmo

AES-256 en ECB deja ver el escudo; el mismo AES-256 en CBC no deja ver nada.

02

Cifrar no es proteger

Confidencialidad sin integridad permite manipulación controlada. Por eso GCM es la respuesta por defecto.

03

El cifrado de disco protege un solo escenario

El servidor apagado. Encendido, los once que consultan la base lo ven todo en claro.

04

Tres preguntas bastan para auditar

Qué algoritmo, qué modo, de dónde sale el vector.

// HOY CAMBIAMOS DE FAMILIA

Todo lo del jueves era sobre **ocultar**. Hoy pasamos a lo contrario: mecanismos que no ocultan nada y cuyo único trabajo es **demostrar** que algo no fue alterado y que viene de quien dice venir.

LO QUE MOSTRÓ LA ENTREGA 2

Veinte minutos de realimentación colectiva sobre el análisis de datos en reposo y respaldos. Estos son los cuatro puntos donde más equipos tropezaron.

// SE CONFUNDIÓ AMENAZA CON CONTROL

Varios informes listan controles sin decir contra qué amenaza sirven.

La tabla del jueves existe justamente para eso: una fila por amenaza, y en cada una si el control la detiene o no.

// SE RECOMENDÓ SIN COSTO NI PLAZO

«Implementar cifrado a nivel de campo» no es una recomendación: es un deseo.

Falta quién lo ejecuta, cuánto tarda y qué dice el contrato del core, que exige cotización para tocar el esquema.

// NO SE MENCIONÓ EL MENSAJERO

El disco de respaldo viaja los viernes a Sabaneta con la llave dentro.

Es el escenario más fácil de explicar ante un comité y el que menos apareció.

// SE OLVIDÓ LA DISPONIBILIDAD

Casi nadie evaluó qué pasa si la llave de respaldos se pierde.

Cifrar mal expone el dato; cifrar sin custodia lo destruye. Las dos cosas son hallazgos.

SECCIÓN

02

FUNCIONES RESUMEN

La huella digital de un dato: qué promete y cuáles ya no cumplen

QUÉ ES UNA FUNCIÓN RESUMEN

Toma una entrada de cualquier tamaño y produce una salida de tamaño fijo. No es cifrado: no hay llave y no hay vuelta atrás. Su trabajo no es ocultar, es identificar.

TAMAÑO FIJO, SEA CUAL SEA LA ENTRADA

«Coopaburrá» → SHA-256 → 8f4a... (32 bytes)
El documento completo de 14 páginas → SHA-256 → 3c91... (32 bytes)
2,3 TB de base de datos → SHA-256 → b70e... (32 bytes)

Siempre 32 bytes. Siempre el mismo resultado para la misma entrada.

// PARA QUÉ SIRVE

Verificar que un archivo no cambió.

Identificar contenido sin revelarlo.

Construir estructuras de integridad encadenadas, como los registros que Coopaburrá no tiene.

// PARA QUÉ NO SIRVE

No es cifrado: no se puede recuperar la entrada a partir del resumen.

No autentica: cualquiera puede recalcularlo, así que no prueba quién lo produjo.

Y por sí sola no protege contraseñas, como vamos a ver en el segundo bloque.

LAS TRES PROMESAS QUE DEBE CUMPLIR

PROPIEDAD	QUÉ SIGNIFICA	SI SE ROMPE...
Resistencia a preimagen	Dado un resumen, no se puede encontrar una entrada que lo produzca	Se podría deducir el contenido a partir de la huella
Resistencia a segunda preimagen	Dado un documento, no se puede fabricar otro con el mismo resumen	Se podría sustituir un contrato por otro sin que la verificación lo note
Resistencia a colisión	No se pueden encontrar dos entradas cualesquiera con el mismo resumen	Se pueden preparar dos documentos con la misma huella y firmar uno para usar el otro

// LA QUE CAE PRIMERO Y LA QUE MÁS DUELE

La resistencia a colisión es la más frágil de las tres porque el atacante elige **las dos** entradas. Es la que se rompió en MD5 y en SHA-1, y es la que importa para firma digital: si puedo fabricar dos contratos con la misma huella, la firma sobre uno vale para el otro.

POR QUÉ 256 BITS Y NO 128

Encontrar una colisión no exige probar todas las salidas posibles: basta con la raíz cuadrada de ese número. Es la llamada paradoja del cumpleaños, y reduce a la mitad los bits de seguridad.

FUNCIÓN	TAMAÑO DEL RESUMEN	ESFUERZO PARA UNA COLISIÓN	ESTADO
MD5	128 bits	En teoría 2^{64} ; en la práctica, segundos	Roto. No usar bajo ninguna circunstancia
SHA-1	160 bits	En teoría 2^{80} ; demostrado en la práctica en 2017	Roto. Retirado de todos los usos de seguridad
SHA-256	256 bits	2^{128} , fuera de alcance	Vigente. Es la opción por defecto
SHA-3 / SHA-512	256 a 512 bits	2^{128} o más	Vigentes. Diseño interno distinto al de SHA-2

// LA REGLA PRÁCTICA

Un resumen de **n** bits ofrece **n/2** bits de seguridad frente a colisiones. Por eso 128 bits de resumen ya no bastan y 256 es el mínimo razonable hoy.

CÓMO MURIERON MD5 Y SHA-1

AÑO	QUÉ PASÓ
2004	Se publica el primer método práctico para producir colisiones en MD5. La comunidad recomienda abandonarlo.
2008	Un equipo demuestra que con colisiones de MD5 se puede fabricar un certificado de autoridad certificadora falso, aceptado por los navegadores.
2012	Un programa malicioso de alto perfil usa una colisión de MD5 para hacerse pasar por software firmado por un fabricante legítimo. Deja de ser un ejercicio académico.
2017	Se publica la primera colisión práctica de SHA-1: dos documentos PDF distintos con el mismo resumen. El cálculo fue costoso pero real.
2020	Se logra una colisión con prefijo elegido en SHA-1, mucho más peligrosa, y a un costo al alcance de un atacante con recursos moderados.
Hoy	Ambas siguen apareciendo en sistemas heredados. Coopaburrá guarda las contraseñas de sus 142.000 asociados con SHA-1.

EL EFECTO AVALANCHA

Cambiar un solo carácter de la entrada cambia aproximadamente la mitad de los bits de salida. No hay parecido parcial: dos entradas casi iguales producen resúmenes sin ninguna relación visible.

DEMOSTRACIÓN EN VIVO

```
echo -n "Coopaburra" | openssl dgst -sha256
echo -n "Coopaburrá" | openssl dgst -sha256      # una sola tilde de diferencia
```

Las dos salidas no se parecen en nada. No existe «casi igual» en un resumen.

// POR QUÉ IMPORTA ESTA PROPIEDAD

Es lo que permite usar un resumen para detectar alteraciones: no hace falta comparar el documento entero, basta comparar 32 bytes. Y si alguien cambió una coma, se nota igual que si hubiera cambiado el archivo completo.

PARA QUÉ SIRVE Y PARA QUÉ NO SIRVE UN RESUMEN

Una función resumen es una herramienta de propósito general, y por eso se usa mal con frecuencia. Conviene tener la lista a mano al revisar cualquier diseño.

// USOS LEGÍTIMOS

Verificar que un archivo descargado es exactamente el publicado.

Detectar cambios en un expediente o en un registro.

Identificar un contenido sin compararlo completo: deduplicación, control de versiones.

Construir cadenas y árboles que protegen la integridad de conjuntos enteros de datos.

// USOS INDEBIDOS

Guardar contraseñas sin función de derivación: el hallazgo H1.

Autenticar sin llave: cualquiera puede recalcular el resumen.

«Anonimizar» **cédulas resumiéndolas**. Hay menos de diez mil millones de cédulas posibles: se calculan todas en segundos y el resumen se revierte por simple búsqueda.

// EL ÚLTIMO ES UN HALLAZGO FRECUENTE EN COLOMBIA

Bases «anonimizadas» para analítica o para terceros con la cédula resumida. Frente a la Ley 1581 ese dato sigue siendo personal, porque se puede volver a identificar. **Resumir un dato de espacio pequeño no lo anonimiza: lo disfraza.**

CÓMO SE CONSTRUYE UNA FUNCIÓN RESUMEN

No hace falta implementarla, pero sí conocer la forma, porque explica un ataque que casi nadie conoce y que justifica por qué el HMAC existe.

CONSTRUCCIÓN ENCADENADA • MD5, SHA-1 Y SHA-2

El mensaje se parte en bloques y se procesa uno tras otro:

estado inicial → [bloque 1] → estado → [bloque 2] → estado → ... → resumen

El resumen final NO es otra cosa que el estado interno al terminar.

// LA CONSECUENCIA INCÓMODA

Si el resumen es el estado interno, quien lo conoce puede continuar el cálculo desde ahí.

Puede añadir contenido al final del mensaje y producir el resumen correcto del mensaje ampliado, sin conocer el principio.

// CÓMO SE LLAMA Y A QUIÉN AFECTA

Se llama ataque de extensión de longitud.

Afecta a MD5, SHA-1 y SHA-2 por su construcción encadenada.

SHA-3 usa un diseño distinto, de esponja, y es inmune por construcción.

SHA-2 Y SHA-3: DOS DISEÑOS PARA LA MISMA PROMESA

SHA-3 no reemplaza a SHA-2: es una alternativa con un diseño interno completamente distinto, escogida en un concurso público en 2012 precisamente para no depender de una sola construcción.

CRITERIO	SHA - 2	SHA - 3
Construcción	Encadenada: bloque tras bloque, como MD5 y SHA-1	Esponja: absorbe el mensaje y luego exprime la salida
Extensión de longitud	Vulnerable, por construcción	Inmune, por construcción
Rendimiento en software	Muy alto, con aceleración en los procesadores modernos	Algo menor en software; muy eficiente en hardware
Salida de longitud variable	No	Sí: SHAKE128 y SHAKE256 entregan tantos bits como se pidan
Uso recomendado	La opción por defecto hoy	Diversificación, y pieza interna de los estándares post-cuánticos

// PARA EL INVENTARIO

SHA-256 es la respuesta correcta en la gran mayoría de los casos. **El hallazgo no es usar SHA-2 en lugar de SHA-3: es usar SHA-2 pegando una llave al mensaje**, que es el error de la diapositiva siguiente.

POR QUÉ HMAC NO ES SIMPLEMENTE RESUMEN DE LLAVE Y MENSAJE

La forma intuitiva de meter un secreto en un resumen es pegarlo delante del mensaje. Es intuitiva y es insegura, precisamente por el ataque de extensión de longitud.

LA RAZÓN DE LA CONSTRUCCIÓN ANIDADA

Ingenuo : `resumen( llave || mensaje )`
un atacante que vea ese valor puede añadir texto al mensaje
y calcular el resumen correcto del resultado, sin saber la llave.

HMAC : `resumen( llave⊕relleno_externo || resumen( llave⊕relleno_interno || mensaje ) )`
dos pasadas anidadas: el estado interno nunca queda expuesto.

// LA LECCIÓN GENERAL

HMAC no es una ocurrencia: es una construcción diseñada y demostrada para resistir un ataque concreto. **Cuando alguien arma su propio esquema de autenticación pegando la llave al mensaje, casi siempre está reinventando la versión insegura.** Es la regla del martes de la sesión 1 aplicada aquí.

COMPARAR TAMBIÉN TIENE QUE HACERSE BIEN

Un detalle de implementación que arruina un HMAC correcto: cómo se comparan los dos valores al verificar.

// COMPARACIÓN INGENUA

La comparación normal de cadenas se detiene en el primer byte que no coincide.

Eso significa que tarda un poquito más cuando los primeros bytes son correctos.

Un atacante que mida esos tiempos puede ir descubriendo el valor byte por byte.

// COMPARACIÓN EN TIEMPO CONSTANTE

Recorre siempre los dos valores completos y acumula las diferencias.

Tarda lo mismo coincida o no, así que no filtra información.

Todas las bibliotecas serias traen una función para esto. Usarla es gratis; no usarla es un hallazgo.

SECCIÓN

03

INTEGRIDAD NO ES AUTENTICIDAD

Por qué un resumen no prueba nada por sí solo, y hasta dónde llega el HMAC

UN RESUMEN NO AUTENTICA NADA

Es el malentendido más común del tema. Publicar el resumen de un archivo junto al archivo no prueba que el archivo sea legítimo: solo prueba que coincide consigo mismo.

// EL RAZONAMIENTO EQUIVOCADO

«Publico el archivo y su resumen. Si alguien lo altera, el resumen no coincide y nos damos cuenta.»

Suena razonable y es falso.

// LO QUE REALMENTE PASA

Quien altera el archivo también recalcula el resumen y publica el nuevo.

La función resumen es pública: cualquiera puede calcularla.

Sin un secreto de por medio, el resumen detecta accidentes —un archivo corrupto, un error de transmisión— pero no detecta a un adversario.

// LA DISTINCIÓN QUE HAY QUE FIJAR HOY

Un resumen da **integridad frente a accidentes**. Para tener integridad frente a un adversario hace falta meter un secreto en la operación, y ahí aparecen dos caminos distintos: el código de autenticación de mensajes y la firma digital.

HMAC: UN RESUMEN CON LLAVE

Combina el mensaje con una llave secreta antes y después de resumirlo. El resultado solo lo puede calcular —y verificar— quien tenga la llave.

LA DIFERENCIA ESTÁ EN EL SECRETO

resumen simple :	SHA-256(mensaje)	→ cualquiera lo calcula
HMAC	: HMAC-SHA256(llave, mensaje)	→ solo quien tiene la llave

Quien recibe el mensaje recalcula el HMAC con su copia de la llave y compara. Si coincide: el mensaje no fue alterado y viene de alguien que tiene la llave.

// FÍJENSE EN LA ÚLTIMA FRASE

«Alguien que tiene la llave». No dice quién. Esa imprecisión, que parece un detalle de redacción, es exactamente el problema de Coopaburrá.

HMAC PASO A PASO SOBRE UNA TRANSFERENCIA

Así funciona exactamente el mecanismo del hallazgo H2. Vale la pena seguirlo con una operación concreta, porque el defecto no está en ningún paso: está en quién tiene la llave.

EL MECANISMO DE H2, OPERACIÓN POR OPERACIÓN

Mensaje: origen=4471|destino=9920|monto=2500000|hora=2026-01-14T09:12:05

Llave: la de la aplicación, igual en todos los teléfonos (evidencia D)

Etiqueta: HMAC-SHA256(llave, mensaje) → 64 caracteres hexadecimales

El servidor recibe mensaje y etiqueta, recalcula con la misma llave y compara.

Si coinciden: el mensaje no cambió Y lo produjo alguien que tiene la llave.

«Alguien que tiene la llave» = cualquiera que haya descargado la aplicación.

// DÓNDE ESTÁ EL DEFECTO

Cada paso es correcto: HMAC-SHA256 es un buen mecanismo y la comparación puede hacerse bien. **El defecto es que la llave no identifica a nadie**, porque la tienen 142.000 teléfonos y cualquiera que descompile el paquete. Una etiqueta válida prueba integridad, no autoría.

LA LLAVE DE UN HMAC: TAMAÑO, ORIGEN Y PROPÓSITO

Un HMAC es tan fuerte como su llave. Estas son las cinco preguntas que hay que hacerle a cualquier llave de autenticación, con las respuestas del caso al lado.

PREGUNTA	LO CORRECTO	EN COOPABURRÁ
Cuánto debe medir	Al menos lo que mide la salida: 32 bytes para HMAC-SHA256	Veinte caracteres de texto escritos a mano
De dónde sale	De un generador criptográfico	De la imaginación de un programador: «Cb4rr4-2023-HMAC-k3y»
Quién la tiene	Las dos partes que se autentican, y nadie más	Cada teléfono con la aplicación instalada
Para qué se usa	Un solo propósito	Todas las operaciones de todos los usuarios
Cada cuánto cambia	Según su periodo criptográfico, que fija la política	Nunca, desde marzo de 2023

// EL VEREDICTO

Cinco de cinco respuestas incorrectas. **Ni una llave perfecta arreglaría H2**, porque el problema de fondo es que es compartida; pero con estas respuestas, además, la llave es mala en sí misma.

DERIVAR MUCHAS LLAVES DE UN SOLO SECRETO: HKDF

El error 4 de la sesión 2 decía: una llave, un propósito. La objeción inmediata es que eso multiplica las llaves que hay que custodiar. HKDF resuelve la objeción.

EXTRAER Y EXPANDIR • RFC 5869

secreto_maestro = 32 bytes de un generador criptográfico ← el único que se custodia

llave_cifrado = HKDF(secreto_maestro, info="respaldo/cifrado/2026")

llave_autentic = HKDF(secreto_maestro, info="respaldo/autenticacion/2026")

llave_indice = HKDF(secreto_maestro, info="indice-ciego/cedula")

Tres llaves independientes: conocer una no revela las otras ni el secreto maestro.

// LA DIFERENCIA CON LAS FUNCIONES DE CONTRASEÑA

HKDF supone que la entrada ya tiene entropía alta, y por eso es rápida. **Para contraseñas no sirve**: ahí hace falta una función lenta a propósito, que es el bloque 6. Y es HKDF lo que usa el canal seguro para derivar sus llaves de sesión, en la sesión 5.

HMAC FRENTE A FIRMA DIGITAL

Los dos dan integridad y autenticidad. Solo uno da no repudio, y la diferencia está en cuántas partes conocen el secreto.

ASPECTO	CÓDIGO DE AUTENTICACIÓN (HMAC)	FIRMA DIGITAL
Tipo de llave	Una sola llave, compartida entre emisor y receptor	Dos llaves: privada para firmar, pública para verificar
Quién puede generar	Cualquiera de las dos partes	Solo quien tiene la llave privada
Quién puede verificar	Solo quien tiene la misma llave	Cualquiera, con la llave pública
¿Da no repudio?	No. Ambas partes pudieron generarlo	Sí. Solo el titular de la privada pudo
Velocidad	Muy rápido	Entre cientos y miles de veces más lento
Cuándo se usa	Integridad entre dos sistemas que ya confían entre sí	Cuando hace falta probar autoría ante un tercero

DÓNDE APARECE HMAC SIN QUE NADIE LO NOMBRE

HMAC está en muchos más lugares de los que parece. Reconocerlo permite hacerle a cada uno las mismas cinco preguntas sobre su llave que le hicimos a Coopaburrá.

DÓNDE	QUÉ AUTENTICA	LA PREGUNTA DE AUDITORÍA
Los códigos de seis dígitos de una aplicación autenticadora	Que quien ingresa tiene la semilla compartida: el código es un HMAC de la hora	¿Dónde guarda el servidor las semillas, y cifradas con qué?
Los tokens de sesión JWT con algoritmo HS256	Que el token lo emitió el servidor y nadie lo alteró	¿La llave es larga y aleatoria, o una palabra en la configuración?
Las notificaciones entre sistemas	Que la notificación viene realmente del proveedor	¿Se verifica la etiqueta, y en tiempo constante?
Las cookies de sesión firmadas	Que el navegador no modificó el contenido	¿Qué pasa si la llave del servidor se filtra?

// EL PUENTE CON LA SESIÓN 4

En los tokens JWT la librería lee del propio token qué algoritmo usar. Si acepta «ninguno», cualquiera fabrica tokens válidos. **Es el primero de los cinco errores de firma del jueves**, y aparece en auditorías reales.

EL CORREO DE BITLAB, OTRA VEZ

Volvamos al documento 2 del expediente con lo que ya sabemos. La frase clave cabe en una línea.

DOCUMENTO 2 · CORREO DE BITLAB S.A.S. · 27 DE ENERO DE 2026

«...el esquema de firma de operaciones utiliza HMAC-SHA256 con una clave compartida provisionada en tiempo de compilación...»

LO QUE DICE

LO QUE SIGNIFICA TÉCNICAMENTE

«HMAC-SHA256»

Mecanismo correcto y vigente. No está roto. Da integridad y autenticidad

«clave compartida»

Por definición, HMAC no puede dar no repudio: las dos partes pueden generar el mismo código

«provisionada en tiempo de compilación»

La misma llave para los 142.000 asociados, dentro del paquete que cualquiera descarga

«el alcance no incluía gestión de llaves»

Nadie era responsable de la pieza de la que dependía todo el esquema

LA PREGUNTA DEL ABOGADO TIENE RESPUESTA TÉCNICA

El 5 de febrero el apoderado preguntó si existe forma técnica de demostrar que los registros no fueron alterados. Nadie respondió. La respuesta existe y se construye con lo que acabamos de ver.

REGISTROS ENCADENADOS POR RESUMEN

```
Registro 1:  datos_1 + resumen( datos_1 )           → H1
Registro 2:  datos_2 + resumen( datos_2 || H1 )      → H2
Registro 3:  datos_3 + resumen( datos_3 || H2 )      → H3
```

Cada registro incorpora el resumen del anterior. Alterar el registro 2 invalida H2, y con él H3 y todos los que siguen.

// LO QUE ESTO SÍ RESUELVE

Nadie puede modificar un registro del pasado sin recalcularlo toda la cadena posterior.

Una alteración deja de ser invisible: se detecta comparando la cadena.

// LO QUE ESTO NO RESUELVE TODAVÍA

Quien controla el servidor puede recalcularlo la cadena entera y presentarla coherente.

Sigue faltando algo que ate la cadena a un momento en el tiempo y a un tercero que no sea la cooperativa.

EL ÁRBOL DE MERKLE: PROBAR UNO SIN REVISAR TODOS

La cadena de registros protege el orden; el árbol de Merkle permite algo más: demostrar que una operación concreta está en el conjunto sin entregar el conjunto completo.

UN RESUMEN DE RESÚMENES • R = SHA-256

```

      raíz = R(H12 || H34)      ← un solo valor para todo el día
      /      \
    H12 = R(H1 || H2)    H34 = R(H3 || H4)
    /      \      /      \
H1 = R(op1) H2 = R(op2) H3 = R(op3) H4 = R(op4) ← una hoja por operación
  
```

Para probar que op3 está incluida basta entregar H4 y H12: con eso se recalcula la raíz. Con 142.000 operaciones, la prueba ocupa apenas 18 resúmenes.

// LO QUE ESTO LE DA A COOPABURRÁ

Si cada noche se sella la raíz del día con un sello de tiempo de un tercero, **todas las operaciones de ese día quedan protegidas con un solo sello**, y cualquiera se puede probar después ante un juez. Es el mismo mecanismo de los registros públicos de certificados de la sesión 5.

LO QUE FALTA PARA QUE SIRVA ANTE UN JUEZ

Un registro encadenado es necesario pero no suficiente. Para que sea oponible a un tercero hacen falta dos piezas más, y las dos aparecen en las sesiones siguientes.

PIEZA	QUÉ APORTA	DÓNDE LA VEMOS
Firma digital de la cadena	Ata la cadena a una llave privada que solo una parte posee, de modo que el servidor no pueda rehacerla a voluntad	Sesión 4
Sello de tiempo de un tercero	Un tercero independiente certifica que ese resumen existía en ese momento. Impide rehacer el pasado	Sesión 6
Custodia y segregación	Que quien opera el sistema no sea quien custodia la llave con la que se firma	Sesión 5 y 6

// GUARDEN ESTA DIAPOSITIVA PARA EL INFORME

Cuando en la sustentación les pregunten cómo se prueba que una operación la autorizó su titular, la respuesta completa son **estas tres piezas más la firma individual por usuario**. Ninguna sola basta.

COOPABURRÁ ELIGIÓ UN MECANISMO QUE NUNCA PUDO DAR LO QUE EL ABOGADO NECESITABA.

Aunque la llave hubiera estado perfectamente guardada, aunque fuera distinta para cada usuario y se rotara cada mes, HMAC seguiría sin dar no repudio, porque el servidor también la conoce y por tanto también pudo generar la operación. El problema no admite arreglo por configuración: exige cambiar de mecanismo. Eso es el jueves.

SECCIÓN

04

CONTRASEÑAS

Bloque 6 · Por qué nunca se cifran, nunca se resumen a secas, y qué hay que hacer

UNA CONTRASEÑA NUNCA SE CIFRA

Primera regla, y la más contraintuitiva: el sistema no necesita conocer la contraseña. Solo necesita poder comprobar si la que le acaban de dar es la correcta.

// SI SE CIFRA

Existe una llave que permite recuperarla. Quien obtenga esa llave obtiene las 142.000 contraseñas en claro.

Y como la gente repite contraseñas, obtiene además el acceso al correo y al banco de mucha de esa gente.

Es exactamente lo que pasó en la filtración de 2013 que vimos el jueves.

// SI SE RESUME

No hay llave y no hay vuelta atrás. Ni siquiera la organización puede recuperar la contraseña.

Para verificar, se resume lo que el usuario acaba de escribir y se compara con lo almacenado.

Por eso los sistemas serios no pueden «recordarle» su contraseña: solo pueden dejarle poner otra.

// UNA SEÑAL DE ALARMA GRATUITA

Si un sistema es capaz de enviarle su contraseña actual por correo, está guardándola de forma recuperable. Es un hallazgo que se detecta sin ninguna herramienta.

RESUMIR TAMPOCO BASTA

Resumir es necesario pero no suficiente, por dos razones que se combinan y que están las dos en el hallazgo H1 de Coopaburrá.

PROBLEMA	POR QUÉ	CONSECUENCIA
Las funciones resumen son rápidas a propósito	SHA-256 está diseñado para procesar gigabytes por segundo	Una tarjeta gráfica prueba miles de millones de contraseñas por segundo
Las personas eligen mal	El espacio real de contraseñas humanas es minúsculo comparado con el teórico	Un diccionario de unos pocos millones cubre la mayoría de las contraseñas reales
Sin sal, el resumen es una huella pública	El resumen de «123456» es siempre el mismo en todo el mundo	Existen tablas ya calculadas: no hay que probar nada, solo buscar

// EL PRIMER HALLAZGO VISIBLE DE LA EVIDENCIA B

Los resúmenes de la muestra tienen 40 caracteres hexadecimales. Eso son 160 bits, que es exactamente el tamaño de SHA-1. **Se puede identificar el algoritmo sin acceso al código, solo contando caracteres.**

LA SAL: QUÉ RESUELVE Y QUÉ NO

La sal es un valor aleatorio que se añade a la contraseña antes de resumirla, y se guarda junto al resultado. No es secreta. Su trabajo es que dos contraseñas iguales produzcan resúmenes distintos.

ESQUEMA	DOS USUARIOS CON LA MISMA CONTRASEÑA	ATAQUE MASIVO
Sin sal	Mismo resumen. Se ve a simple vista en la tabla	Una sola pasada del diccionario rompe toda la base a la vez
Sal única para toda la base	Mismo resumen. No mejora nada en este punto	Una sola pasada, solo que con la sal añadida. Igual de barato
Sal distinta por usuario	Resúmenes distintos. No se puede saber quién repite contraseña	Hay que atacar cada cuenta por separado: el costo se multiplica por 142.000

// POR QUÉ LA SAL DE COOPABURRÁ NO SIRVE

Una sal constante para toda la base **no es una sal**: es un sufijo. Solo protege contra tablas precalculadas genéricas, y ni siquiera eso, porque basta calcular una tabla nueva con esa constante. No protege contra el ataque que de verdad importa: romper las 142.000 cuentas en una sola pasada.

LA SAL, CON NÚMEROS: LA MISMA CONTRASEÑA, DOS RESULTADOS

Dos asociados del caso, usuario_a y usuario_c, eligieron la misma contraseña. Lo que la base guarda de cada uno depende de si hay sal, y de qué clase.

EVIDENCIA B • DOS USUARIOS QUE COMPARTEN RESUMEN

CON sal única por usuario – la misma contraseña, resúmenes distintos

usuario_a • "password" + sal 8f2a...c1 → argon2id → e7b3a9d0...

usuario_c • "password" + sal a17d...04 → argon2id → f4c09122... ← distinto: no se nota que repiten

SIN sal, como el portal hoy (evidencia B) – la misma contraseña, el MISMO resumen

usuario_a • "password" → sha1 → 5baa61e4c9b93f3f0682250b6cf8331b7ee68fd8

usuario_c • "password" → sha1 → 5baa61e4c9b93f3f0682250b6cf8331b7ee68fd8 ← idéntico

// LO QUE LA SAL SÍ HACE, Y LO QUE NO

La sal no esconde la contraseña —de eso se encarga la función lenta—. La sal esconde **que dos personas usan la misma**, y obliga al atacante a trabajar cuenta por cuenta en vez de romper toda la base en una sola pasada. En Coopaburrá, como la sal ni siquiera se aplica, ese resumen es el valor público de «password».

CÓMO SE USA LA SAL, Y CÓMO SE USA MAL

La sal es fácil de usar bien y fácil de arruinar. Solo el primer patrón funciona.

CÓMO SE USA	¿SIRVE?	POR QUÉ
Aleatoria y única por usuario, junto al resumen	Sí	Lo correcto: cada cuenta se ataca por separado y el costo se multiplica por 142.000
Constante para toda la base	No	Un sufijo, no una sal: una tabla con esa constante rompe todo. Es Coopaburrá
Derivada del usuario, como la cédula	No	Es predecible: el atacante la conoce y precalcula igual que sin sal
Demasiado corta, de pocos bits	Débil	Se precalculan tablas para todas las sales posibles. Usen 16 bytes
Reutilizada entre usuarios	No	Dos con la misma sal y contraseña vuelven a colisionar

// LA REGLA DE ORO

Una sal por registro, aleatoria, de al menos 16 bytes y guardada junto al resumen. **Si dos registros comparten sal, ya falló.**

EL TALLER DEL ATACANTE: CUÁNTO CUESTA CADA INTENTO

Cifras aproximadas de pruebas públicas con una sola tarjeta gráfica de gama alta. No importa el número exacto: importa la diferencia de orden de magnitud entre las filas.

FUNCIÓN	INTENTOS POR SEGUNDO	TODAS LAS DE OCHO MINÚSCULAS · $2,1 \times 10^{11}$
MD5	Unos 160.000 millones	Un segundo
SHA-1 · la de Coopaburrá	Unos 50.000 millones	Cuatro segundos
SHA-256	Unos 20.000 millones	Diez segundos
bcrypt, costo 10	Unos 5.000	Más de un año
bcrypt, costo 12	Unos 1.400	Más de cuatro años

// LO QUE CAMBIA DE VERDAD

Pasar de SHA-1 a una función con factor de trabajo multiplica el costo de cada intento por unos diez millones. **Ningún cambio de política de contraseñas consigue ni una fracción de eso.** Por eso es la primera recomendación de H1, antes que cualquier regla nueva. Las funciones que exigen memoria son todavía peores para el atacante.

FUNCIONES DE DERIVACIÓN DE LLAVE

La solución real no es resumir mejor: es resumir **lento a propósito**. Una función de derivación repite la operación miles de veces y consume memoria deliberadamente, para que cada intento le cueste al atacante.

FUNCIÓN	AÑO	QUÉ HACE COSTOSO	RECOMENDACIÓN
PBKDF2	2000	Repite la operación muchas veces. Solo consume tiempo	Aceptable si es lo único disponible por normativa
bcrypt	1999	Tiempo y algo de memoria. Límite de longitud de entrada	Aceptable, ampliamente probado
scrypt	2009	Tiempo y bastante memoria	Buena opción
Argon2id	2015	Tiempo, memoria y paralelismo, los tres calibrables	La recomendación actual para sistemas nuevos

// POR QUÉ IMPORTA LA MEMORIA Y NO SOLO EL TIEMPO

Una tarjeta gráfica tiene miles de núcleos pero poca memoria por núcleo. Una función que exige mucha memoria por intento anula esa ventaja y deja al atacante sin su mejor herramienta. Por eso las funciones modernas piden memoria, no solo repeticiones.

ARGON2ID: LOS PARÁMETROS Y CÓMO SE GUARDAN

La recomendación actual para sistemas nuevos, con los valores de referencia de las dos fuentes que se citan en cualquier auditoría: el RFC que la estandariza y la guía de OWASP.

PARÁMETRO	QUÉ CONTROLA	VALOR DE REFERENCIA
Memoria · m	Cuánta RAM exige cada intento: lo que anula la ventaja de la tarjeta gráfica	19 MiB como mínimo según OWASP; 64 MiB en la segunda opción del RFC 9106
Iteraciones · t	Cuántas pasadas se hacen sobre esa memoria	2 con 19 MiB; 3 con 64 MiB
Paralelismo · p	Cuántos hilos usa el servidor legítimo	1 según OWASP; 4 según el RFC
Sal	Única por usuario, de un generador criptográfico	16 bytes
Salida	Tamaño del resultado almacenado	32 bytes

// POR QUÉ LOS PARÁMETROS SE GUARDAN CON EL RESULTADO

Cada registro se guarda como «\$argon2id\$v=19\$m=19456,t=2,p=1\$sal\$resultado». **Así se pueden subir los parámetros el año próximo sin romper los registros viejos:** cada uno se verifica con los suyos y se actualiza en el siguiente ingreso del usuario.

CÓMO SE ELIGE EL FACTOR DE TRABAJO

No hay un número universal: depende del servidor y de cuánta espera tolera el usuario. La regla de calibración es sencilla y se puede defender ante un comité.

01 Fije el tiempo objetivo por verificación

Entre 0,25 y 1 segundo en el servidor de producción es el rango habitual para un portal.

02 Mida con el hardware real, no con su portátil

El factor correcto en el servidor de Itagüí no es el mismo que en la máquina de desarrollo.

03 Calcule el impacto en el pico de carga

El portal tiene 9.000 sesiones diarias. Hay que verificar que el costo sea sostenible en la hora pico.

04 Documente el parámetro y revíselo cada año

El hardware del atacante mejora. Un factor calibrado en 2026 será insuficiente en 2030.

LA PIMIENTA: UN SECRETO QUE NO VIVE EN LA BASE

La sal no es secreta y viaja con el resumen. La pimienta es lo contrario: un secreto de la aplicación, común a todos los registros, que nunca se guarda en la base de datos.

// CÓMO FUNCIONA

El resultado de Argon2id se pasa por un HMAC con la pimienta antes de guardarlo.

La pimienta vive en un servicio de gestión de llaves o en un módulo de hardware, nunca en el código ni en la base.

Un robo de la base sola, por inyección de SQL o por un respaldo extraviado, entrega resúmenes que no se pueden atacar sin ella.

// LO QUE EXIGE

Vivir separada de la base: si están juntas, no aporta nada.

Un plan de rotación: cambiarla obliga a conservar la anterior mientras los registros se actualizan.

Y no reemplaza nada: es una capa sobre la sal por usuario y la función lenta, nunca un sustituto.

// LA IRONÍA DEL CASO

Coopaburrá tiene algo parecido a una pimienta: la constante «c00pab2019», común a todos los registros. **El problema es que vive en el código fuente, a la vista de once personas**, que es exactamente donde una pimienta no puede estar.

LO QUE CAMBIÓ EN LAS POLÍTICAS DE CONTRASEÑA

Las reglas que casi todas las organizaciones siguen fueron desaconsejadas por la propia entidad que las había recomendado. Coopaburrá exige seis caracteres y no obliga a cambiarlas: está mal en un extremo, pero la corrección no es irse al otro.

PRÁCTICA TRADICIONAL	RECOMENDACIÓN ACTUAL	POR QUÉ CAMBIÓ
Exigir mayúscula, número y símbolo	No imponer reglas de composición	Producen contraseñas predecibles: la gente escribe la misma palabra con un 1 y un signo al final
Obligar a cambiarla cada 90 días	Cambiar solo ante indicio de compromiso	El cambio forzado empeora las contraseñas: se vuelven variaciones numeradas de la anterior
Mínimo de 8 caracteres	Permitir frases largas y no limitar el máximo	La longitud aporta mucha más entropía que la complejidad
Sin verificación externa	Contrastar contra listas de contraseñas ya filtradas	Es la medida con mejor relación entre costo y efecto, y Coopaburrá no la tiene

CONSULTAR CONTRASEÑAS FILTRADAS SIN REVELARLAS

La medida más rentable de la tabla anterior tiene un problema aparente: para saber si una contraseña está filtrada habría que enviarla a un tercero. Una idea elegante con funciones resumen lo resuelve.

CONSULTA POR RANGO · K-ANONIMATO

1. Se calcula el SHA-1 de la contraseña que el asociado quiere usar:
5BAA61E4C9B93F3F0682250B6CF8331B7EE68FD8
2. Se envían al servicio SOLO los primeros cinco caracteres: 5BAA6
3. El servicio devuelve todos los resúmenes filtrados que empiezan así:
unos pocos cientos de sufijos, con cuántas veces apareció cada uno
4. La comparación final se hace en el servidor propio: el tercero nunca sabe cuál de esos cientos era, ni si alguno lo era.

// EL DETALLE QUE CONVIENE NOTAR

Aquí SHA-1 es aceptable: no protege nada, solo sirve de índice. **La misma función que es un error para guardar contraseñas es correcta para buscarlas**, y distinguir los dos usos es el criterio que se evalúa.

SEIS COSAS QUE UN SISTEMA DE CONTRASEÑAS NUNCA DEBE HACER

Errores que se detectan desde afuera, sin acceso al código, y que casi siempre anuncian problemas más graves adentro.

PRÁCTICA	QUÉ REVELA O QUÉ PROVOCA
Escribir la contraseña en los registros de la aplicación	Pasa más de lo que parece: queda en claro en los archivos de depuración, fuera de todo control
Limitar la longitud máxima a pocos caracteres	Suele delatar que se guarda en un campo fijo, a veces en claro. E impide las frases largas
Usar preguntas de seguridad para recuperar	Un segundo secreto más débil que el primero: el nombre de la madre está en las redes sociales
Responder distinto a usuario inexistente y a contraseña incorrecta	Permite averiguar qué usuarios existen antes de atacarlos
No limitar ni frenar los intentos fallidos	Hace gratis el ataque en línea, que con seis minúsculas es suficiente
Recortar la contraseña sin avisar	Solo cuentan los primeros caracteres. bcrypt, por ejemplo, ignora lo que pase de 72 bytes

QUÉ SE GUARDA POR CADA USUARIO

El rediseño del hallazgo H1 se concreta en esta tabla. Es lo que debería tener la base de Coopaburrá y lo que hoy no tiene.

CAMPO	QUÉ ES	¿SECRETO?	EN COOPABURRÁ HOY
Identificador	El usuario o el documento	No	Existe
Sal	Valor aleatorio distinto para cada usuario, generado al crear la cuenta	No, pero única	Constante para todos
Algoritmo y parámetros	Qué función de derivación y con qué factor de trabajo se calculó	No	No se almacena
Resultado de la derivación	La salida de la función aplicada a contraseña y sal	Sí	Resumen SHA-1 simple
Fecha del último cambio	Para detectar cuentas antiguas al migrar	No	Por verificar

// POR QUÉ SE GUARDAN LOS PARÁMETROS JUNTO AL RESULTADO

Porque el factor de trabajo va a cambiar con los años. Si está almacenado con cada registro, se pueden convivir varios factores durante la migración y subirlos poco a poco. Si no está, cambiar el parámetro invalida toda la base de un golpe.

EL PROBLEMA DE MIGRAR 142.000 CONTRASEÑAS

Este es el punto donde casi todos los proyectos se atascan, y es parte de la entrega 3. No lo voy a resolver aquí: lo van a resolver ustedes.

// EL CALLEJÓN

Para guardar las contraseñas con la función nueva hay que conocerlas.

Pero no se conocen: solo hay resúmenes SHA-1, y de un resumen no se vuelve atrás.

Y pedirle a 142.000 asociados que cambien su contraseña el mismo día no es viable ni comercial ni operativamente.

// LAS PISTAS

¿Es obligatorio partir de la contraseña en claro, o se puede partir de lo que ya está almacenado?

¿Puede convivir la base con dos esquemas al tiempo durante un periodo?

¿Qué momento es el único en que el sistema sí ve la contraseña en claro, aunque sea por un instante?

// LO QUE SE EVALÚA DE ESTA PARTE

Que la solución no exija conocer las contraseñas actuales, que no obligue a un cambio masivo el mismo día, y que diga qué pasa con las cuentas que no vuelven a entrar nunca. Esa última es la que casi nadie contempla.

EL SEGUNDO FACTOR CAMBIA LA ECUACIÓN

Todo lo anterior mejora la resistencia de la contraseña. Un segundo factor cambia el problema de sitio, y para una entidad vigilada no es opcional.

FACTOR	QUÉ APORTA	LÍMITE
Código por mensaje de texto	Mejor que nada y muy adoptado	Vulnerable al secuestro de línea telefónica, que en Colombia ocurre
Aplicación generadora de códigos	No depende de la red telefónica	El código se puede pedir por engaño en tiempo real
Llave física o credencial del dispositivo	Vinculada al sitio: no se puede entregar por engaño a un sitio falso	Costo y logística de distribución
Vinculación con el dispositivo	Una llave distinta por teléfono, generada en el propio dispositivo	Es justamente lo que le habría faltado a la aplicación de Coopaburrá

// LA CONEXIÓN CON EL HALLAZGO H2

La última fila no es un complemento: es parte de la solución del hallazgo. Una llave por dispositivo, generada en el dispositivo y de la que el servidor solo conoce la parte pública, es el camino hacia la firma individual. Eso es exactamente la sesión del jueves.

LLAVES DE ACCESO: EL RELEVO DE LA CONTRASEÑA

El sector se está moviendo a un esquema en el que no hay contraseña que guardar ni que robar. Conviene entenderlo, porque es exactamente el patrón que resuelve el hallazgo H2.

FIDO2 Y WEBAUTHN . CÓMO FUNCIONA

Registro: el teléfono genera una pareja de llaves para ESE sitio.
La privada queda en el enclave del dispositivo; el sitio guarda la pública.

Ingreso: el sitio envía un desafío aleatorio.
El dispositivo lo firma, previa huella o rostro del usuario.
El sitio verifica la firma con la llave pública que tiene guardada.

No hay secreto compartido: robar la base del sitio no sirve para entrar.

// POR QUÉ RESISTE EL ENGAÑO

La llave queda atada al dominio del sitio: **en un sitio falso, el dispositivo simplemente no tiene llave que usar**, y el usuario no puede entregarla aunque quiera. Es la defensa técnica más sólida contra la suplantación, y la firma que la sostiene es el tema del jueves.

RECUPERACIÓN DE CUENTA: LA PUERTA QUE SE OLVIDA

Un sistema de autenticación es tan fuerte como su procedimiento de recuperación. Es la puerta por la que entran los atacantes cuando la principal está bien cerrada.

VÍA DE RECUPERACIÓN	CÓMO SE ATACA	QUÉ LA HACE DEFENDIBLE
Código por mensaje de texto	Duplicado fraudulento de la tarjeta SIM en un punto de venta del operador	Combinarla con otro factor y avisar por un canal distinto
Llamada al centro de contacto	Ingeniería social con datos filtrados: cédula, fecha de nacimiento, dirección	Preguntas que no estén en ninguna filtración y una espera antes de aplicar el cambio
Correo electrónico	Tomar primero el correo, que suele estar peor protegido	Que cambiar el correo exija el factor fuerte
Presencial en la oficina	Documentos falsificados	Validación biométrica de la identidad

// LA REGLA

La recuperación nunca debe ser más fácil que el ingreso normal. **Y todo cambio de credenciales se notifica por un canal distinto al que se usó para hacerlo,** con un plazo durante el cual el titular puede detenerlo.

SECCIÓN

05

LABORATORIO 3

Cincuenta minutos · Atacar la evidencia B, con el ejercicio en Python

LOS TRES EJERCICIOS

Trabajan sobre la muestra real de la tabla de usuarios del caso, evidencia B del expediente. Está anonimizada: los usuarios no existen, pero los resúmenes son auténticos.

01 Ejercicio 1 · Identificar el algoritmo sin ver el código

A partir de la longitud del resumen, determinen qué función lo produjo. Justifiquen con el número de bits.

02 Ejercicio 2 · Romper la muestra

Dos de los tres usuarios tienen el mismo resumen. Averigüen qué contraseñas son y expliquen cómo llegaron: qué les dijo la repetición y qué papel jugó la sal declarada.

03 Ejercicio 3 · Medir la diferencia

Calculen cuánto tarda una función de derivación moderna frente a un resumen simple para el mismo intento, y extrapolen: ¿cuánto costaría atacar 142.000 cuentas en cada esquema?

// UNA ADVERTENCIA SOBRE EL EJERCICIO 2

Van a descubrir algo que el expediente no dice del todo bien, y encontrarlo es parte del ejercicio. **Documenten la discrepancia entre lo que la documentación afirma y lo que los datos demuestran.** Eso es exactamente el trabajo de un auditor.

PUNTO DE PARTIDA

LABORATORIO 3 • DATOS Y COMANDOS

```
# Evidencia B • muestra de la tabla de usuarios del portal
1 usuario_a 5baa61e4c9b93f3f0682250b6cf8331b7ee68fd8
2 usuario_b 7c4a8d09ca3762af61e59520943dc26494f8941b
3 usuario_c 5baa61e4c9b93f3f0682250b6cf8331b7ee68fd8

# sal declarada en la documentación: constante «c00pab2019»

# calcular un resumen y comparar
echo -n "candidata" | openssl dgst -sha1
echo -n "candidatac00pab2019" | openssl dgst -sha1
```

// LA PREGUNTA QUE ORDENA TODO EL EJERCICIO

¿Cuál de las dos órdenes de arriba reproduce los resúmenes de la tabla? La respuesta decide si la sal se está aplicando o solo está documentada, y esa diferencia es un hallazgo con nombre propio.

LABORATORIO 3 - EL EJERCICIO EN PYTHON

Los ejercicios 2 y 3 hechos con la biblioteca estándar: confirmar si la sal declarada se aplicó, y medir cuánto cuesta un intento con factor de trabajo.

LABORATORIO 3 · PYTHON 3, SOLO BIBLIOTECA ESTÁNDAR

```
import hashlib, time
objetivo = "5baa61e4c9b93f3f0682250b6cf8331b7ee68fd8" # usuario_a y usuario_c
for clave in open("comunes.txt", encoding="utf-8").read().split():
    if hashlib.sha1(clave.encode()).hexdigest() == objetivo:
        print("coincide SIN sal:", clave)
    if hashlib.sha1((clave + "c00pab2019").encode()).hexdigest() == objetivo:
        print("coincide CON la sal declarada:", clave)

t = time.perf_counter()
hashlib.scrypt(b"candidata", salt=b"sal-unica-16-bytes", n=2**14, r=8, p=1)
print(round(time.perf_counter() - t, 3), "segundos por intento")
```

// LO QUE VA EN EL INFORME

¿Con cuál de las dos condiciones coincidió? Esa sola línea confirma o desmiente la documentación de la fábrica de software. **Y extrapolen: ¿cuántos años tomaría recorrer la misma lista si cada intento costara lo que costó scrypt?**

PRIMERA EVALUACIÓN DE CONOCIMIENTO

Veinte minutos, individual, sin material. Cubre las sesiones 1 a 3 y vale el 15 % de la nota del módulo. Estos son los temas, para que nadie estudie de más ni de menos.

TEMA	QUÉ HAY QUE SABER HACER
Los cuatro servicios	Dada una situación, decir cuál falló y con qué mecanismo se habría evitado
Entropía y generación de llaves	Distinguir un generador adecuado de uno que no lo es, y estimar entropía real
Modos de operación	Elegir el modo correcto y justificarlo. Reconocer el problema de ECB y el de repetir el vector
Los tres estados del dato	Decir qué protege cada mecanismo y contra qué amenaza concreta
Resúmenes y HMAC	Diferenciar integridad, autenticidad y no repudio, y saber cuál da cada mecanismo
Contraseñas	Explicar por qué no se cifran, qué hace la sal y para qué sirve el factor de trabajo

LO QUE LLEVAMOS DE LA SESIÓN

01

Un resumen no autentica a nadie

Detecta accidentes, no adversarios. Cualquiera puede recalcularlo.

02

MD5 y SHA-1 están rotos y siguen en producción

Coopaburrá guarda 142.000 contraseñas con SHA-1.

03

HMAC da autenticidad pero nunca no repudio

Porque las dos partes conocen la llave. Ningún ajuste lo cambia.

04

Una contraseña no se cifra ni se resume a secas

Se deriva con una función lenta a propósito, con sal única por usuario.

05

La sal constante de Coopaburrá no es una sal

Es un sufijo, y no impide el ataque que de verdad importa.

TERCERA PARTE DEL PRODUCTO

Se entrega al inicio de la sesión 4: rediseño del almacenamiento de contraseñas y de la firma de operaciones. Hallazgos H1 y H2.

SECCIÓN	QUÉ DEBE CONTENER
Diagnóstico de H1	Qué algoritmo se usa, qué permite exactamente y qué demostró el laboratorio sobre la sal declarada
Rediseño del almacenamiento	Función elegida, parámetros propuestos y cómo se calibraron. Con nombre de responsable
Plan de transición	No se pueden reconvertir 142.000 contraseñas sin conocerlas. ¿Cómo se migra sin pedirle a todo el mundo que cambie la suya el mismo día?
Diagnóstico de H2	Por qué el mecanismo actual no puede dar no repudio, con el argumento técnico, no solo la conclusión

// LA PARTE DIFÍCIL ES LA TERCERA FILA

Migrar contraseñas es el problema clásico: no se conocen las actuales, así que no se pueden volver a derivar. Hay una solución estándar y elegante, y encontrarla es parte del ejercicio.

TRABAJO INDEPENDIENTE HASTA LA SESIÓN 4

9 h

TOTAL ENTRE
MARTES Y JUEVES

3 h

LECTURA
PREVIA

6 h

ENTREGA 3 E INFORME
DEL LABORATORIO 3

Lectura previa · 3 horas

RFC 2104 y RFC 5869

HMAC y HKDF: los dos documentos cortos que definen cómo se autentica un mensaje y cómo se derivan llaves. Una hora.

Katz y Lindell (2020)

El capítulo de gestión de llaves y la revolución de la llave pública: Diffie-Hellman y el problema que resuelve. 1,5 horas.

Ley 527 de 1999

Artículos 2, 5 a 13 y 28: mensaje de datos, firma y firma digital. Media hora. Se discute el jueves en el bloque 8.

// CONDICIÓN DE ENTRADA A LA SESIÓN 4

El jueves se asume leído el artículo 28 de la Ley 527: la discusión del bloque 8 parte de sus cinco atributos. Las 6 horas de elaboración son la entrega 3 (5 h, en equipo) y el informe del laboratorio (1 h).

LO QUE VIENE EL JUEVES

Sesión 4: criptografía de llave pública y firma digital. Es la sesión que resuelve el problema que hoy solo diagnosticamos.

// BLOQUE 1 • DOS LLAVES EN LUGAR DE UNA

El problema del intercambio de llaves y cómo lo resuelve la llave pública.

RSA y curvas elípticas: por qué las segundas desplazaron a la primera.

Firma digital: el único mecanismo que da no repudio, y por qué.

// BLOQUE 2 • EL VALOR PROBATORIO

La Ley 527 y qué hace que una firma electrónica sea confiable ante un juez.

El hallazgo H7: veinte años de hipotecas firmadas con una imagen escaneada.

Laboratorio: firmar, verificar y comparar el costo de RSA frente a curvas elípticas.

SIGLAS DE LA SESIÓN · 1 DE 2

AES / ECB / CBC / GCM

Advanced Encryption Standard y sus modos: Electronic Codebook, Cipher Block Chaining, Galois/Counter Mode. Sesión 2.

FIDO2 / WebAuthn

Fast IDentity Online 2 / Web Authentication — Estándares de las llaves de acceso: una pareja de llaves por sitio.

FIPS

Federal Information Processing Standards — Normas federales de procesamiento de información de Estados Unidos, del NIST.

HKDF

HMAC-based Key Derivation Function — Deriva varias llaves de un secreto de alta entropía. RFC 5869.

HMAC

Hash-based Message Authentication Code — Código de autenticación de mensajes basado en una función resumen. RFC 2104.

IETF / RFC

Internet Engineering Task Force / Request for Comments — El organismo de estándares de internet y su serie de documentos.

MD5

Message Digest 5 — Función resumen de 128 bits de 1992, rota por colisiones desde 2004.

MiB / RAM / TB

Mebibyte, 2²⁰ bytes / Random Access Memory, memoria de trabajo / Terabyte. Argon2id se calibra en MiB de RAM.

NIST / SP

National Institute of Standards and Technology / Special Publication — El instituto de normas de Estados Unidos y su serie técnica.

OWASP

Open Worldwide Application Security Project — Fundación que publica guías abiertas de seguridad de aplicaciones.

Toda sigla utilizada en la sesión aparece en el glosario con su forma completa en el idioma original y su equivalencia en español.

SIGLAS DE LA SESIÓN · 2 DE 2

PDF

Portable Document Format — Formato de documento. La colisión de SHA-1 de 2017 se demostró con dos PDF distintos.

RSA

Rivest, Shamir y Adleman — Algoritmo de llave pública de 1977. Sesión 4.

SHA-1 / SHA-2 / SHA-256

Secure Hash Algorithm — Funciones resumen del NIST. SHA-256 es la variante de 256 bits de la familia SHA-2.

SHA-3 / SHAKE128 / SHAKE256

La familia de 2015 basada en la construcción de esponja. SHAKE entrega salidas de la longitud que se pida.

SIM

Subscriber Identity Module — La tarjeta que identifica una línea móvil. Su duplicado fraudulento permite recibir los códigos de otro.

SQL

Structured Query Language — Lenguaje de bases de datos. Una inyección de SQL puede extraer la base completa.

CRYPTO / LNCS

Conferencia internacional anual de criptografía / Lecture Notes in Computer Science, la serie donde se publican sus actas.

JWT / HS256

JSON Web Token / HMAC con SHA-256 — Formato de token de sesión y su variante autenticada con llave compartida.

Toda sigla utilizada en la sesión aparece en el glosario con su forma completa en el idioma original y su equivalencia en español.

REFERENCIAS DE LA SESIÓN

NIST. (2015).	FIPS 180-4: Secure Hash Standard y FIPS 202: SHA-3 Standard.	Las familias SHA-2 y SHA-3
Krawczyk, H., Bellare, M. y Canetti, R. (1997).	RFC 2104: HMAC: Keyed-hashing for message authentication. IETF.	La construcción anidada · lectura previa de la sesión 4
Krawczyk, H. y Eronen, P. (2010).	RFC 5869: HMAC-based extract-and-expand key derivation function (HKDF). IETF.	Derivar varias llaves de un secreto
Biryukov, A. et al. (2021).	RFC 9106: Argon2 memory-hard function for password hashing and proof-of-work applications. IETF.	Parámetros recomendados
Stevens, M. et al. (2017).	The first collision for full SHA-1. CRYPTO 2017, LNCS 10401, 570–596.	La colisión práctica de SHA-1
OWASP. (s. f.).	Password Storage Cheat Sheet. OWASP Cheat Sheet Series.	Argon2id, pimienta y migración de resúmenes heredados
NIST. (revisión vigente).	SP 800-63B: Digital identity guidelines — Authentication.	Verificadores memorizados: longitud, listas de bloqueo y rotación

// ACCESO A LAS FUENTES

Las publicaciones del NIST, los RFC y la guía de OWASP son de acceso libre. El artículo de CRYPTO se consulta en las bases de datos de la Institución.